Unit Code and Title: COS80001 Cloud Engineering, Semester 1, 2025

Report Title: AWS-Based Photo Album Application Deployment using VPC, EC2, RDS, S3, and phpMyAdmin

Student Name: Anmol Srivastava

Student Number: 105026098

Course and Specialization/Major: Master of Information Technology (Professional Computing) - Data Analytics

Workshop Time: 06.30 pm – 08.30 pm, Class 1-14

Facilitator Name: Mr. Naveed Ali

Acknowledgment of Country

We respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

We are honoured to recognise our connection to Wurundjeri Country, history, culture and spirituality through these locations, and strive to ensure that we operate in a manner that respects and honours the Elders and Ancestors of these lands.

We also respectfully acknowledge Swinburne's Aboriginal and Torres Strait Islander staff, students, alumni, partners and visitors.

We also acknowledge and respect the Traditional Owners of lands across Australia, their Elders, Ancestors, cultures and heritage, and recognise the continuing sovereignties of all Aboriginal and Torres Strait Islander Nations.

Table of Contents

Acknowledgment of Country 2
Abstract: 4
1 Introduction: 4
2 VPC AND SUBNET DESIGN: 5
3 EC2 VIRTUAL MACHINES: 6
3.1 WebServerInstance: 6
3.2 Test Instance: 8
4 RDS DATABASE INSTANCE: 8
5 PHPMYADMIN CONFIGURATION: 9
6 NETWORK ACLS: 10
7 PHOTO STORAGE USING S3: 11
8 RDS DATABASE TABLE STRUCTURE: 13
9 WEBSITE CONFIGURATION: 13
10 TESTING: 13
11 CONCLUSION: 14
12 REFERENCES: 15

Abstract:

This paper outlines how to set up and launch a secure web application on Amazon Web Services (AWS) that can handle growth. We explain the process step-by-step, covering the configuration of key AWS components. The architecture uses a custom Virtual Private Cloud (VPC) for network security, EC2 instances for computing power, and a private RDS MySQL database to keep data safe. Network ACLs are implemented to control traffic flow securely. Image files for the application are stored in a publicly reachable S3 bucket. Users interact with the application through a web interface to view these images, while the associated metadata is managed in the RDS database via phpMyAdmin.

1 Introduction:

The development and deployment of applications within cloud computing environments were recognized as requiring thorough and careful planning of the supporting technical infrastructure. This project, therefore, aimed to provide a comprehensive account documenting the process followed to build and subsequently deploy a secure photo album web application hosted entirely on the Amazon Web Services (AWS) platform. Several key objectives were established as priorities throughout this deployment effort. Paramount among these were the implementation of robust security measures across all layers of the architecture to protect application data and user access. Additionally, significant focus was placed on designing the system with scalability in mind, ensuring it possessed the inherent capacity to handle potential future growth in user traffic and data volume. Finally, the entire process was guided by the principle of adhering to established and recommended best practices within the field of cloud computing to ensure an efficient, reliable, and well-managed solution.

2 VPC AND SUBNET DESIGN:

The fundamental network structure required for hosting the web application was meticulously established through the configuration of a custom Virtual Private Cloud (VPC) within the Amazon Web Services ecosystem. This isolated virtual network, specifically named ASrivastavaVPC, was provisioned in the us-east-1 AWS region. It was allocated a substantial private IPv4 address space defined by the Classless Inter-Domain Routing (CIDR) block 10.0.0.0/16, providing ample room for internal IP addressing. A key aspect of the network design was the deliberate incorporation of high availability and fault tolerance; this was achieved by distributing the network infrastructure across two distinct Availability Zones (AZs) located within the us-east-1 region. These AZs represent physically separate data centers, minimizing the risk of simultaneous failure. Within the first designated Availability Zone, two subnets were carved out from the main VPC CIDR block: Public Subnet 1, assigned the range 10.0.1.0/24 and intended for internet-facing resources, and Private Subnet 1, assigned 10.0.2.0/24 for resources requiring isolation from direct internet access. This pattern was precisely mirrored in the second Availability Zone, where Public Subnet 2 (10.0.3.0/24) and Private Subnet 2 (10.0.4.0/24) were similarly created, as illustrated in Figure 2. 1. To facilitate necessary communication with the public internet for resources situated in the public subnets (like the web servers), a specific public route table was configured. This route table was then explicitly associated with both Public Subnet 1 and Public Subnet 2, as illustrated in Figure 2. 2. Critically, this route table contained a default route (0.0.0.0/0) directing outbound internet traffic towards an Internet Gateway (IGW), which had been created and attached to the ASrivastavaVPC to serve as the managed entry and exit point for traffic between the VPC and the external internet, as illustrated in Figure 2. 3.

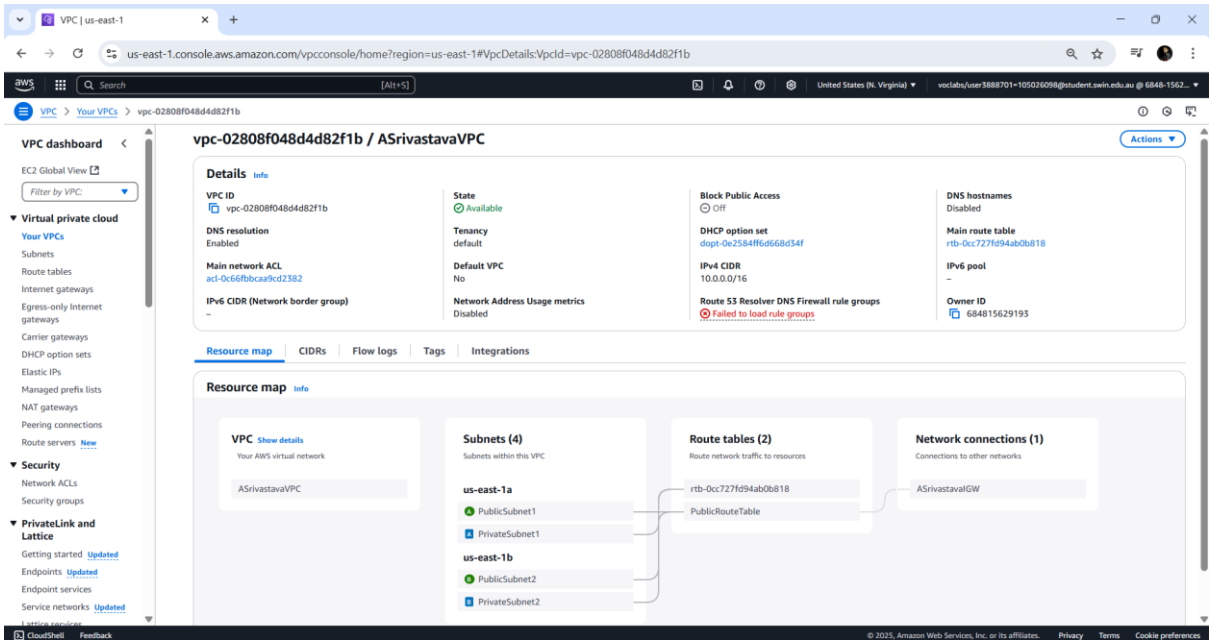


Figure 2. 1

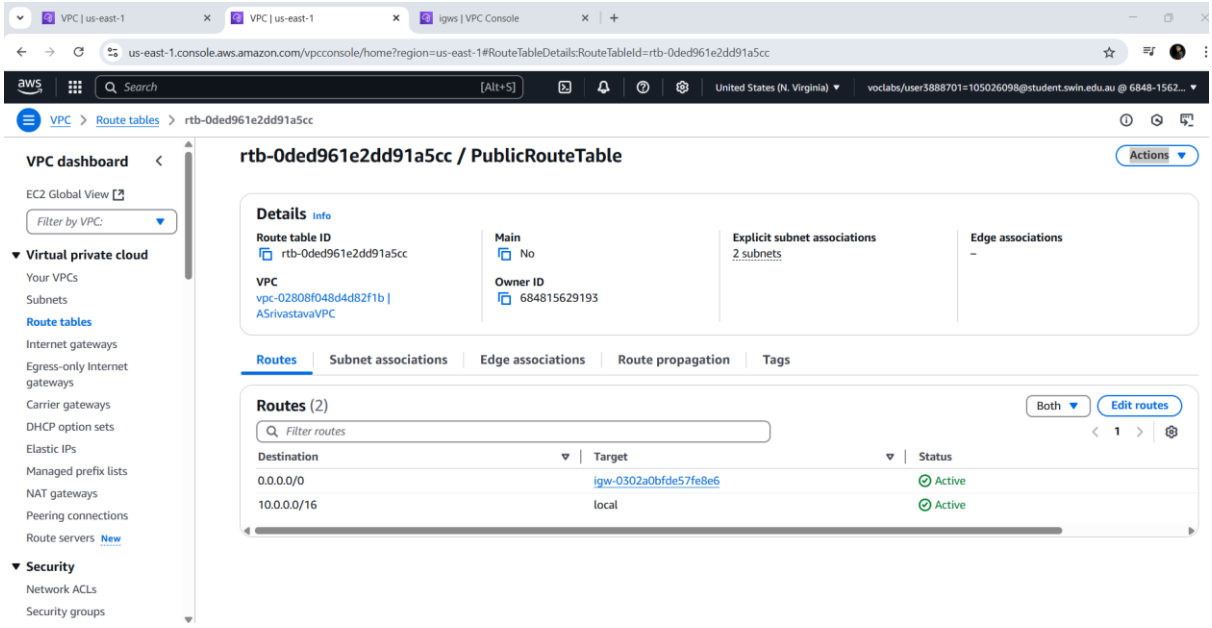


Figure 2. 2

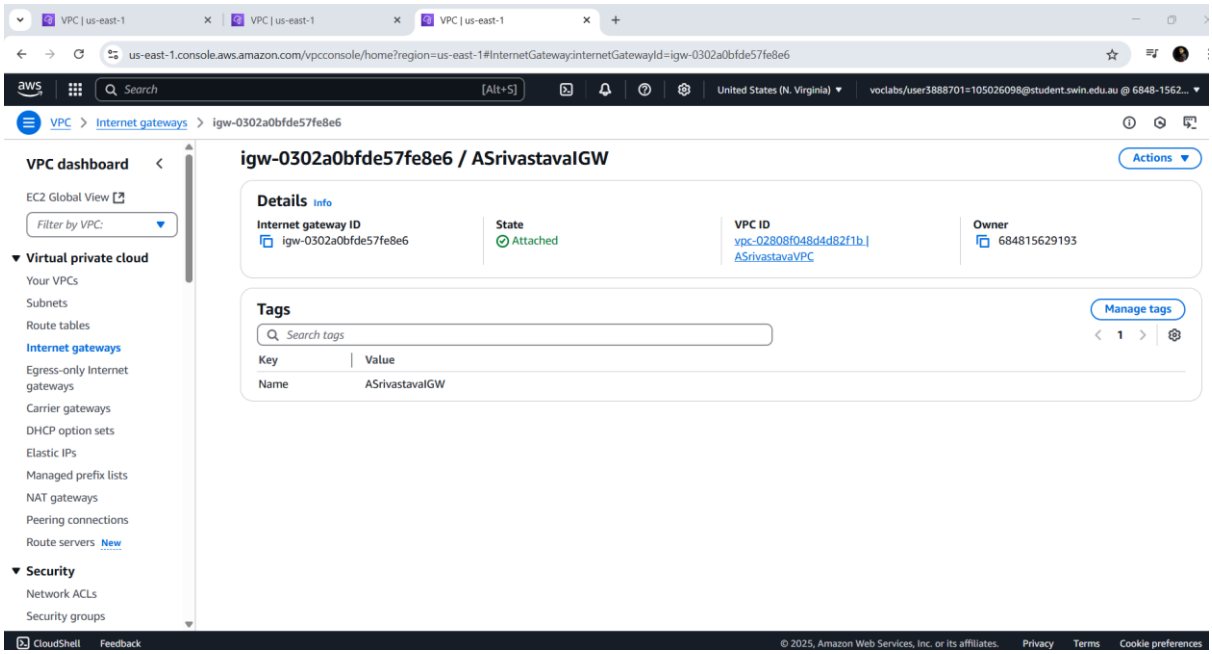


Figure 2. 3

3 EC2 VIRTUAL MACHINES:

To provide the necessary computing resources for the web application and testing, Elastic Compute Cloud (EC2) instances were launched within the configured VPC. Two distinct instances were provisioned for specific roles.

3.1 WebServerInstance:

The primary instance, designated as WebServerInstance, as illustrated in Figure 3. 1, served as the host for the photo album web application itself. For its operating system foundation, the Amazon Linux 2023 Amazon Machine Image (AMI) was selected,

providing a standardized and optimized Linux environment. The t2.micro instance type was chosen for this deployment, offering a balance of compute, memory, and network resources suitable for a small-scale application and potentially qualifying for the AWS Free Tier. Critically, this instance was launched into Public Subnet 2, ensuring it could be assigned a public IP address and be accessible from the internet via the configured Internet Gateway. Secure administrative access via SSH was enabled through the use of the specific key pair file named Assignment-1b.ppk. Following the instance launch, essential web server software components were installed and configured; this included the Apache HTTP Server to serve web content, PHP to process the application's server-side logic, and phpMyAdmin to provide a web-based interface for managing the MySQL database. The photo album application code was then deployed onto this prepared instance. Finally, to ensure a stable and unchanging public endpoint for users accessing the application, an Elastic IP address was allocated within the AWS account and subsequently associated with the WebServerInstance. This provided a static public IP address that persists independently of the instance's lifecycle.

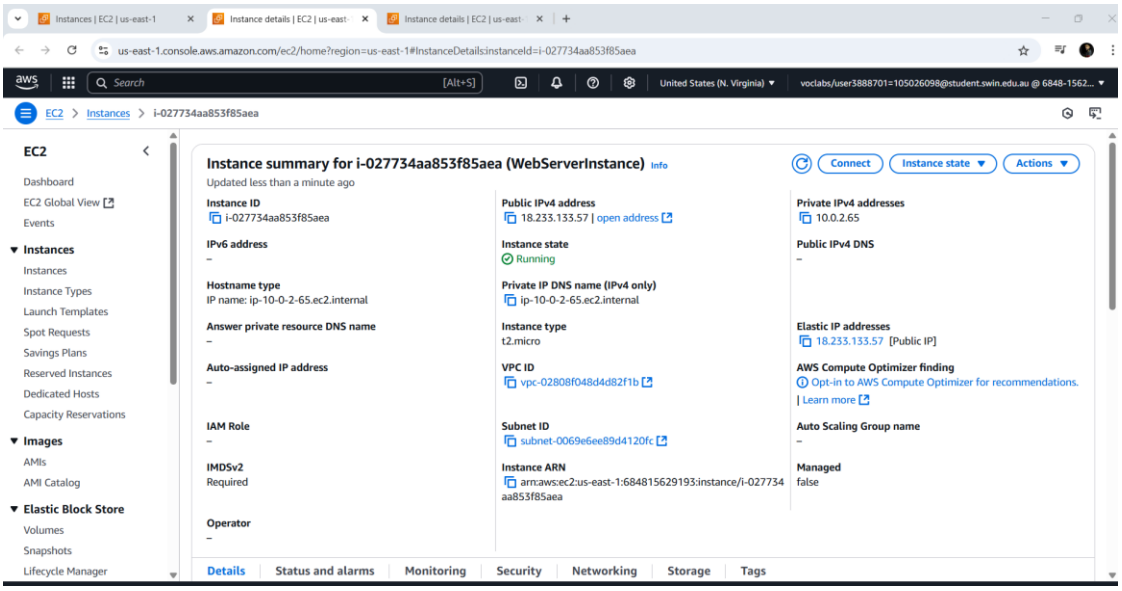
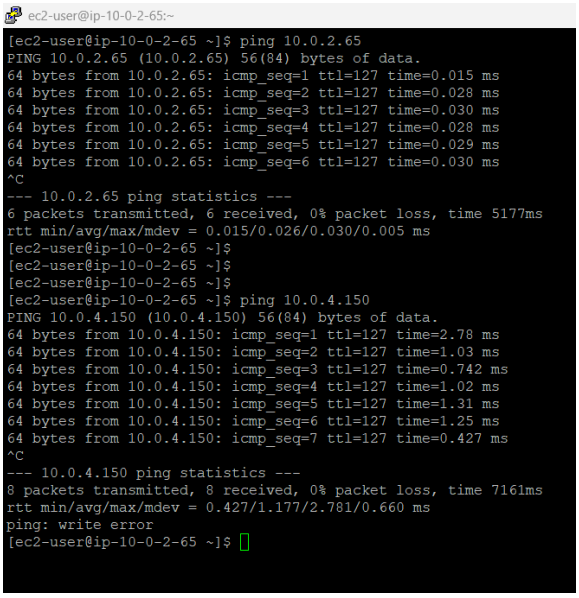


Figure 3. 1



```
ec2-user@ip-10-0-2-65:~
[ec2-user@ip-10-0-2-65 ~]$ ping 10.0.2.65
PING 10.0.2.65 (10.0.2.65) 56(84) bytes of data.
64 bytes from 10.0.2.65: icmp_seq=1 ttl=127 time=0.015 ms
64 bytes from 10.0.2.65: icmp_seq=2 ttl=127 time=0.028 ms
64 bytes from 10.0.2.65: icmp_seq=3 ttl=127 time=0.030 ms
64 bytes from 10.0.2.65: icmp_seq=4 ttl=127 time=0.028 ms
64 bytes from 10.0.2.65: icmp_seq=5 ttl=127 time=0.029 ms
64 bytes from 10.0.2.65: icmp_seq=6 ttl=127 time=0.030 ms
^C
--- 10.0.2.65 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5177ms
rtt min/avg/max/mdev = 0.015/0.026/0.030/0.005 ms
[ec2-user@ip-10-0-2-65 ~]$
[ec2-user@ip-10-0-2-65 ~]$
[ec2-user@ip-10-0-2-65 ~]$
[ec2-user@ip-10-0-2-65 ~]$ ping 10.0.4.150
PING 10.0.4.150 (10.0.4.150) 56(84) bytes of data.
64 bytes from 10.0.4.150: icmp_seq=1 ttl=127 time=2.78 ms
64 bytes from 10.0.4.150: icmp_seq=2 ttl=127 time=1.03 ms
64 bytes from 10.0.4.150: icmp_seq=3 ttl=127 time=0.742 ms
64 bytes from 10.0.4.150: icmp_seq=4 ttl=127 time=1.02 ms
64 bytes from 10.0.4.150: icmp_seq=5 ttl=127 time=1.31 ms
64 bytes from 10.0.4.150: icmp_seq=6 ttl=127 time=1.25 ms
64 bytes from 10.0.4.150: icmp_seq=7 ttl=127 time=0.427 ms
^C
--- 10.0.4.150 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7161ms
rtt min/avg/max/mdev = 0.427/1.177/2.781/0.660 ms
ping: write error
[ec2-user@ip-10-0-2-65 ~]$
```

Figure 3. 2

3.2 Test Instance:

A second EC2 instance, referred to as the Test Instance, as illustrated in Figure 3. 3, was launched specifically for network diagnostic and connectivity verification purposes during the infrastructure setup. Unlike the web server, this instance was intentionally placed within Private Subnet 1. This placement meant it did not have direct inbound or outbound internet access via the Internet Gateway, reflecting a common scenario for backend or internal systems. Its primary utility was confirmed through its use in performing tests such as verifying Secure Shell (SSH) connectivity to other instances within the VPC (like potentially attempting to SSH to the WebServerInstance's private IP) and conducting Internet Control Message Protocol (ICMP) ping tests to check network reachability between different subnets and resources inside the custom VPC environment.

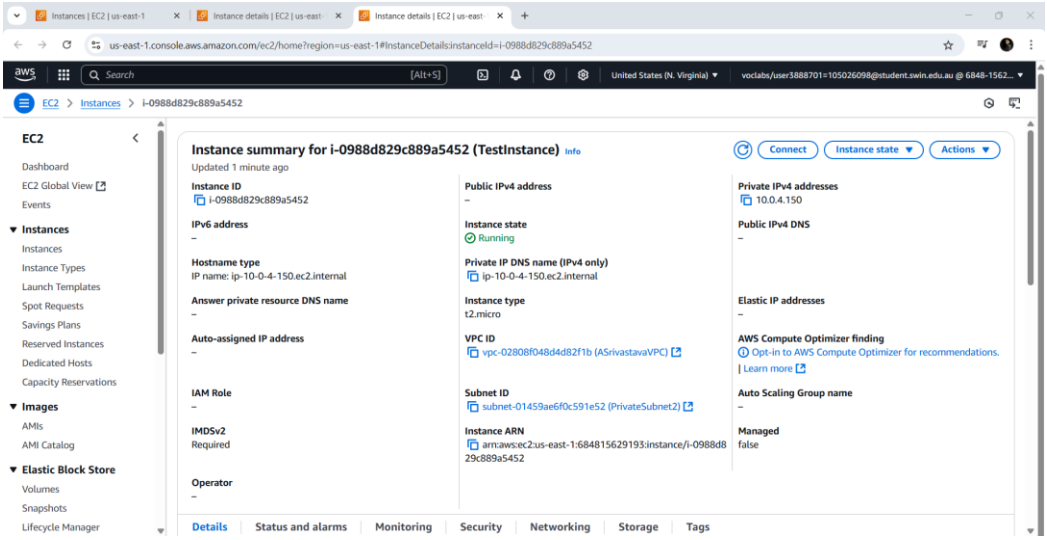


Figure 3. 3

4 RDS DATABASE INSTANCE:

To manage the application's relational data, such as photo metadata, a managed database solution was provisioned using Amazon Relational Database Service (RDS). The specific database engine chosen for this deployment was MySQL, utilizing version 8.0.39. This selection provided a robust, widely-used, and open-source compatible database system suitable for typical web application workloads. To align with potential cost constraints or the scale of this particular project, the Free tier template was selected during the RDS instance configuration. This template offers specific instance types and storage options suitable for development, testing, or small-scale production environments at no charge under the AWS Free Tier limits, as illustrated in Figure 4. 1. A critical security measure implemented was the explicit configuration of Public access to 'No'. This setting ensured that the RDS instance was not assigned a public IP address and could not be reached directly from the internet, significantly reducing its exposure to external threats. Instead, the database was intended to be accessible only from within the custom VPC. To facilitate this internal connectivity and enhance availability, an RDS Subnet Group was created, as illustrated in Figure 4. 2. This group was configured to include both Private Subnet 1 and Private Subnet 2, spanning the two Availability Zones used in the VPC design. This configuration instructed RDS that the database instance should be placed within one of these private subnets, allowing for failover across AZs if needed, while keeping it isolated from public access. Finally, network traffic control for the database instance was enforced by associating it with the previously defined RDSSG Security Group. As detailed earlier, this security group restricted inbound connections exclusively to the MySQL port (3306) and permitted traffic only from sources linked to the WebServerSG, thereby ensuring that only the application's web servers could communicate with the database.

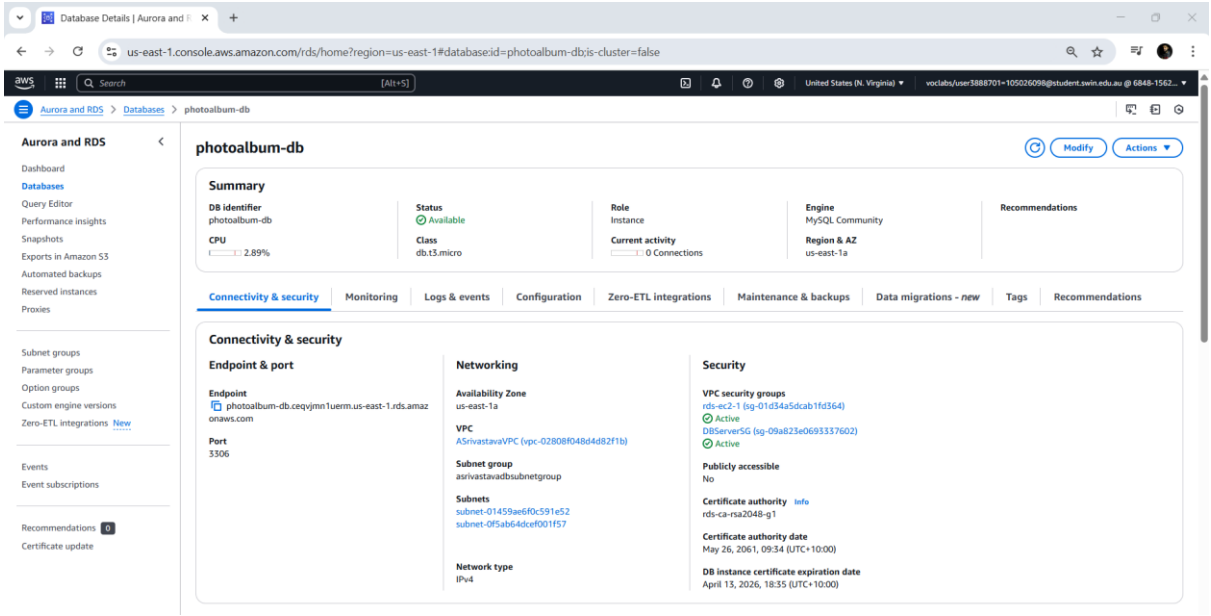


Figure 4. 1

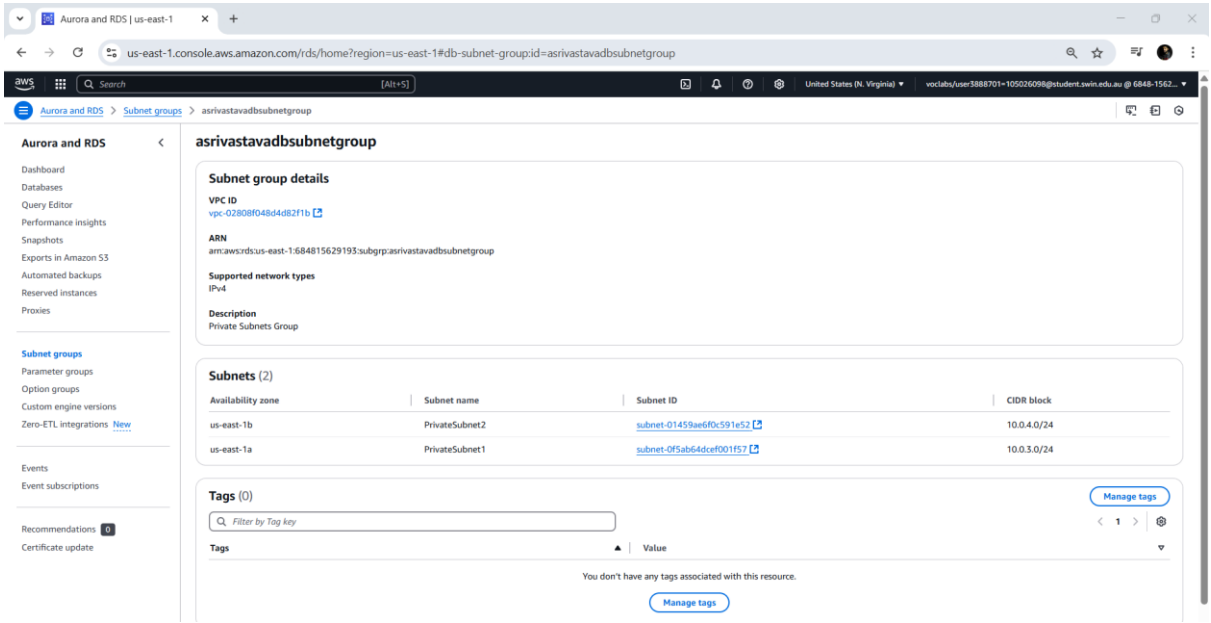


Figure 4. 2

5 PHPMYADMIN CONFIGURATION:

To facilitate convenient web-based administration of the MySQL database hosted on RDS, the phpMyAdmin software package was installed directly onto the WebServerInstance EC2 machine. Following the installation, specific configuration steps were undertaken to establish a connection between phpMyAdmin and the provisioned RDS database. This involved editing the phpMyAdmin configuration files to specify the unique database endpoint address provided by RDS, along with the necessary credentials for authentication. During the setup process, it was observed that standard file system permissions initially prevented the Apache web server from correctly accessing or executing the phpMyAdmin files, leading to access errors when attempting to use the tool via a web browser. To rectify this situation, adjustments were made to the file and directory

permissions within the phpMyAdmin installation directory on the web server. These changes ensured that the web server process had the appropriate read and execute privileges, thereby resolving the access issues and enabling successful operation of the phpMyAdmin interface for database management tasks, as illustrated in Figure 5. 1.

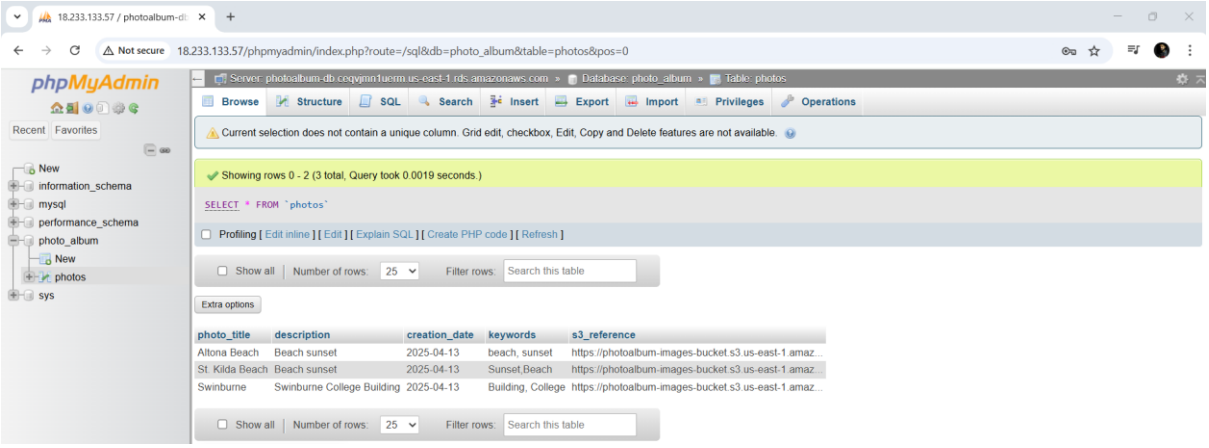


Figure 5. 1

6 NETWORK ACLS:

As an additional layer of network security operating at the subnet level, Network Access Control Lists (NACLs) were configured. NACLs act as stateless firewalls, meaning that rules must be explicitly defined for both inbound and outbound traffic. For this deployment, a specific NACL named PublicSubnet2NACL was created, as illustrated in Figure 6. 1, presumably intended for association with Public Subnet 2, where the WebServerInstance resided. Several specific rules were added to this NACL to control the types of traffic permitted into and potentially out of the associated subnet.

One rule explicitly allowed inbound Secure Shell (SSH) traffic on port 22 from any source IP address, indicated by the CIDR notation 0.0.0.0/0. This configuration permitted administrative access to instances within the subnet from anywhere on the internet. Another rule was configured to permit Internet Control Message Protocol (ICMP) traffic, commonly used for network diagnostics like ping requests. However, this rule was more restrictive, allowing ICMP traffic only if it originated from the IP address range corresponding to Private Subnet 1 (10.0.2.0/24), facilitating internal network connectivity tests from resources within that specific private subnet. Furthermore, rules were added to allow standard web traffic: inbound Hypertext Transfer Protocol (HTTP) on port 80 and inbound Hypertext Transfer Protocol Secure (HTTPS) on port 443 were permitted. This was essential for allowing users' web browsers to connect to the web application hosted on the WebServerInstance. It is important to note that since NACLs are stateless, corresponding outbound rules allowing return traffic would also need to be configured or rely on the default "ALLOW ALL" outbound rule for the NACL to function correctly, although these outbound rules were not detailed in the provided information, as illustrated in Figure 6. 1 and Figure 6. 2.

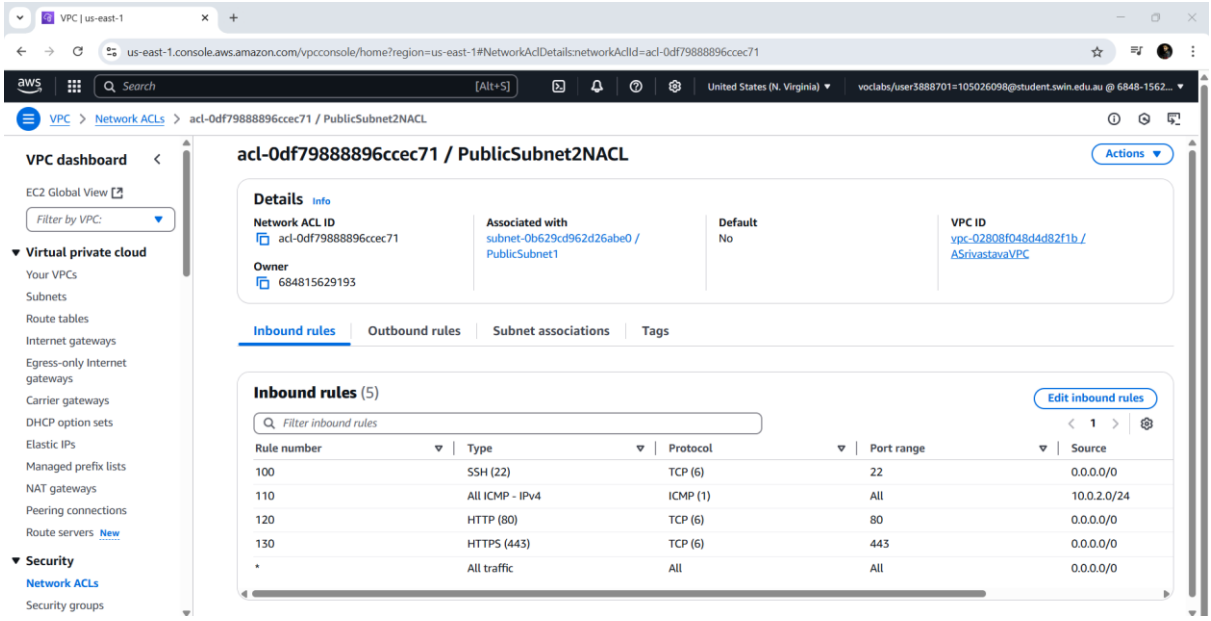


Figure 6. 1

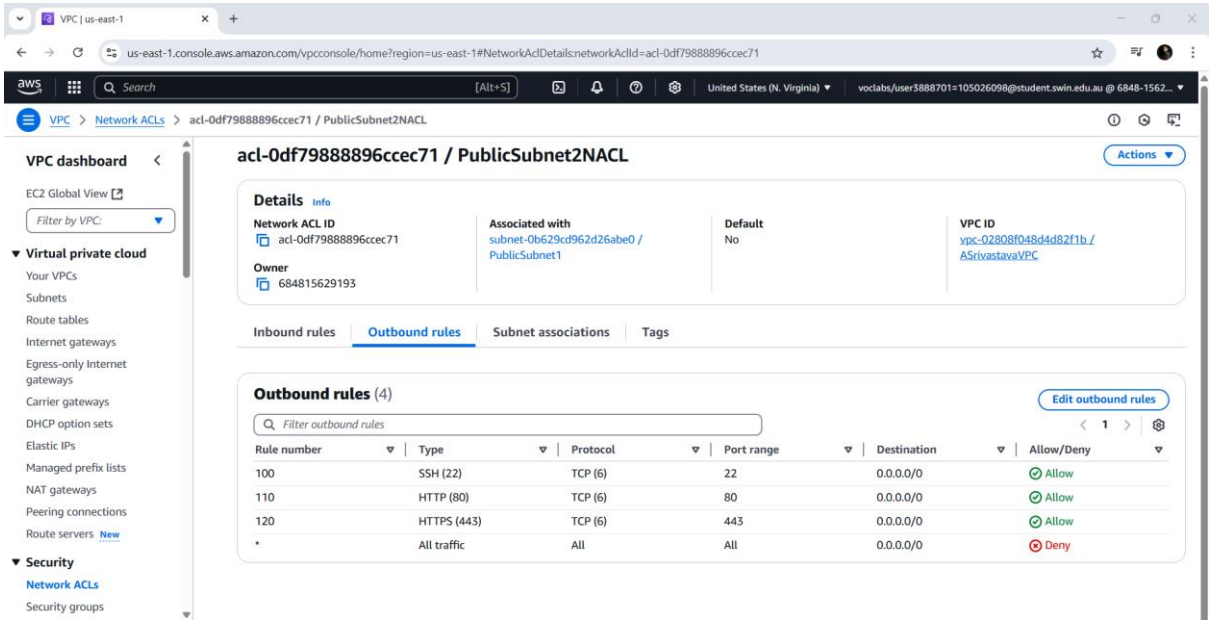


Figure 6. 2

7 PHOTO STORAGE USING S3:

Amazon Simple Storage Service (S3) was selected as the storage solution for the image files associated with the photo album application due to its scalability, durability, and cost-effectiveness. A dedicated S3 bucket was created within the AWS account to serve as the primary repository for these images; for illustrative purposes, this bucket was named photoalbum-images-bucket, as illustrated in Figure 7. 1. To enable the web application to display the photos directly to users via their browsers, a specific configuration allowing public access to the images was necessary. This involved navigating to the bucket's settings within the AWS Management Console and ensuring that the "Block all public access" options were unchecked, thereby permitting the application of a public access policy.

Following this initial step, the access permissions were more precisely defined by applying a custom bucket policy. This was accomplished by accessing the "Permissions" tab for the photoalbum-images-bucket. Within this tab, the "Bucket Policy" section was located, and the editor was used to input a specific policy document formatted in JSON, as illustrated in Figure 7.2. This policy explicitly grants permission ("Effect": "Allow") to any anonymous user ("Principal": "*") to perform the s3:GetObject action, which equates to reading or downloading objects. This permission was specifically scoped to apply to all objects (/*) contained within the specified S3 bucket ("Resource": "arn:aws:s3:::photoalbum-images-bucket/*"). The Sid ("AllowPublicRead") serves as an optional identifier for the policy statement. After applying this policy, the configuration was validated. Sample image files were manually uploaded into the photoalbum-images-bucket, and it was subsequently verified that these images could be successfully accessed and viewed directly via their public S3 URLs in a web browser, confirming that the storage and public access permissions were operating as intended.

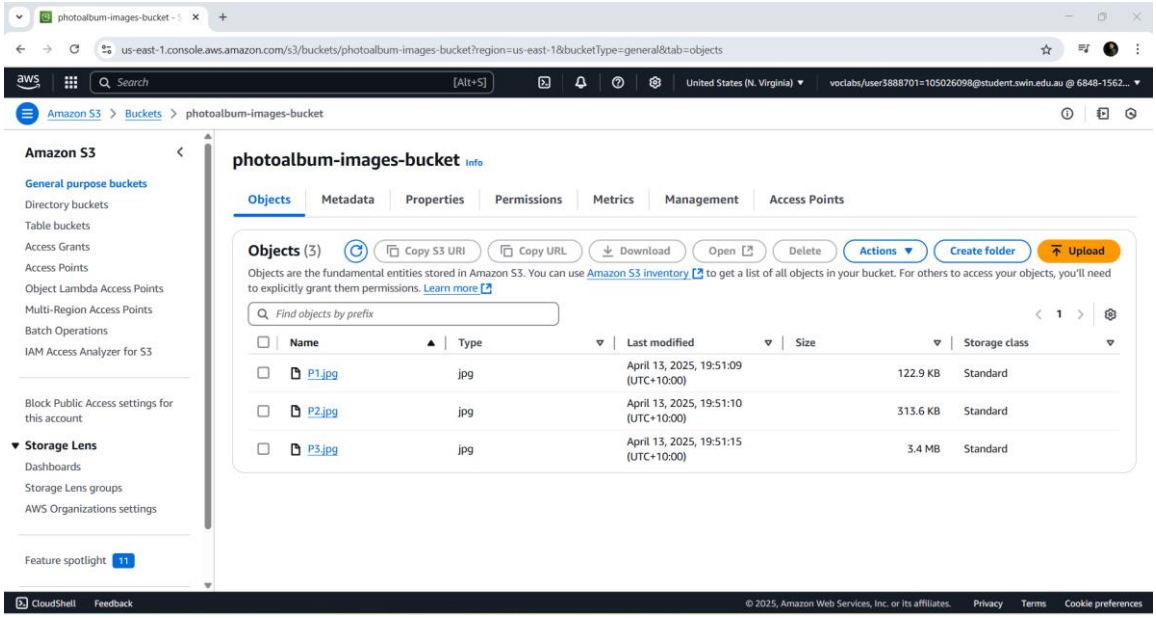


Figure 7. 1

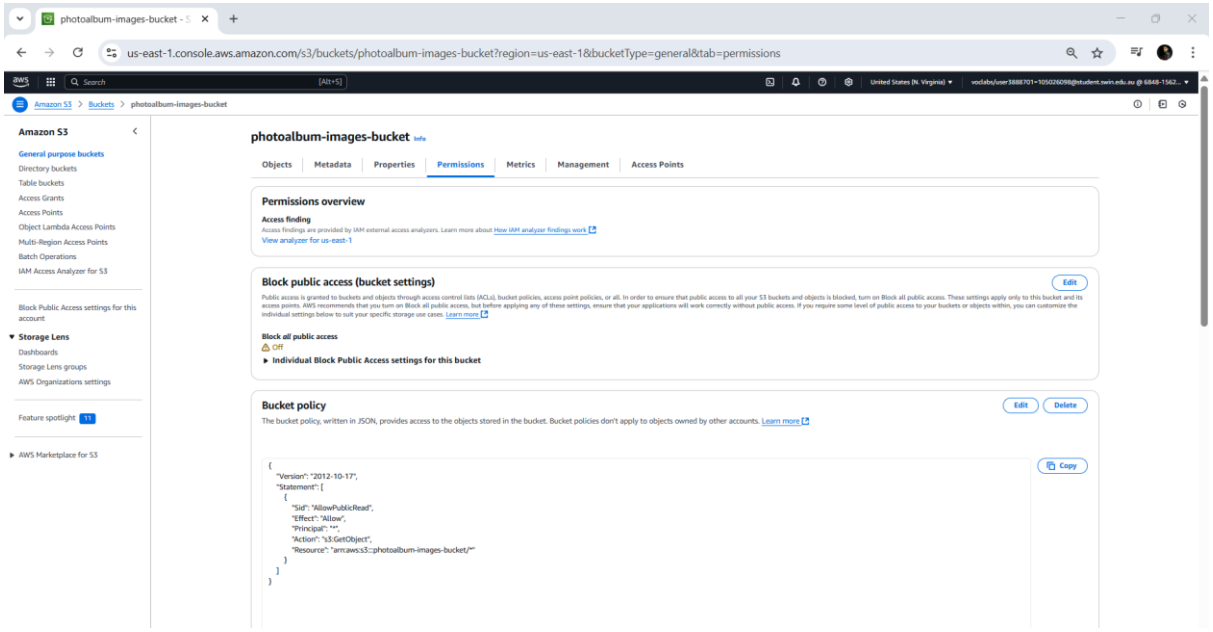


Figure 7. 2

8 RDS DATABASE TABLE STRUCTURE:

Within the provisioned RDS MySQL database instance, a specific table structure was defined to store the metadata associated with the images managed by the photo album application. A single table named photos was created to serve this purpose. This table was designed with several columns, each intended to hold a particular piece of information about the uploaded photos.
A column named photo_title was included, utilizing the VARCHAR(255) data type, suitable for storing textual titles for each photo up to 255 characters in length. Similarly, a description column, also of type VARCHAR(255), was added to allow for brief textual descriptions accompanying each image. To record when a photo entry was created or perhaps when the photo itself was taken, a creation_date column was defined using the DATE data type, appropriate for storing calendar dates. Another textual field, keywords, was implemented as VARCHAR(255) to store relevant keywords or tags associated with each photo, aiding potential search or categorization functionalities. Crucially, a column named s3_reference was established, also using VARCHAR(255). This column was designed to store the necessary information to link the database record to the actual image file stored in the S3 bucket, likely holding the S3 object key or the full object URL, as illustrated in Figure 5. 1.

After defining the structure of the photos table within the MySQL database, initial data population was performed for testing and demonstration purposes. This involved inserting several sample records directly into the photos table. The previously configured phpMyAdmin web interface, running on the WebServerInstance, was utilized as the tool to execute these SQL insertion commands, thereby verifying that the database schema was correctly implemented and accessible for data manipulation via the web server.

9 WEBSITE CONFIGURATION:

Following the setup of the infrastructure components (EC2, RDS, S3) and the installation of necessary software (Apache, PHP, phpMyAdmin), the final step involved configuring the photo album application code itself to interact correctly with these backend services. A key part of this process involved modifying a specific configuration file within the application's codebase, identified as constants.php. This file, likely designed to centralize critical connection parameters, was edited to include the specific endpoint address of the provisioned RDS MySQL database instance and the name or relevant details of the configured S3 bucket used for photo storage. Populating this file with the correct parameters was essential, as it provided the application's backend scripts with the necessary information to establish connections to the database for metadata operations and to construct the correct URLs for accessing image files stored in S3.
Once these application-level configurations were successfully applied and saved on the WebServerInstance, the photo album web application was rendered operational and made accessible to users over the network. The primary entry point for accessing the application was confirmed to be via its specific URL path, which was constructed using the associated Elastic IP address of the WebServerInstance, followed by the designated application path. Specifically, users could access the web application by navigating their web browsers to http://18.233.133.57/cos80001/photoalbum/album.php.

10 TESTING:

With the infrastructure deployed and the application configured, a validation phase was executed to ensure all components functioned correctly and interacted as designed. The primary verification involved accessing the deployed photo album application using its public endpoint. Specifically, navigation to the URL http://18.233.133.57/cos80001/photoalbum/album.php was

performed using a standard web browser. This test successfully confirmed that the web server was responsive and that the application's main page loaded correctly, displaying the photo listings retrieved from the RDS database and rendering the corresponding images fetched from the S3 bucket, along with their associated database (photo_title, description, creation_date, keywords, s3_reference).

Internal network connectivity and security configurations were also explicitly tested. Using the Test Instance situated within Private Subnet 1, network diagnostic commands were executed. It was verified that ICMP ping requests from the Test Instance successfully reached other internal network endpoints (such as the WebServerInstance's private IP), confirming the Network ACL and Security Group rules permitted this type of internal diagnostic traffic. Additionally, SSH connectivity was tested from the Test Instance* towards theWebServerInstance`, successfully establishing a session and thus validating the Security Group rules allowing internal administrative access between designated instances.

These combined tests—spanning application-level functionality, database interaction, image retrieval from S3, and internal network connectivity checks—served to confirm that the website was fully operational and that the underlying AWS infrastructure components were correctly configured and integrated as per the project's design specifications.

11 CONCLUSION:

In summary, this report documented the successful execution of deploying a secure and fully functional web application, specifically a photo album system, utilizing the infrastructure and services provided by Amazon Web Services (AWS). The process detailed herein demonstrated how careful and meticulous planning, particularly concerning the Virtual Private Cloud (VPC) layout with its public and private subnets across multiple Availability Zones, established a robust and isolated network environment. Security, a primary objective, was addressed through a layered approach involving precisely configured access controls; stateful Security Groups were applied at the instance level to strictly regulate traffic (e.g., ensuring the RDS database was only reachable from the web server), while stateless Network ACLs provided an additional boundary at the subnet level.

The successful integration of key AWS managed services was fundamental to the outcome. EC2 provided the necessary compute environment for the web server and application logic, RDS offered a secure and managed relational database solution configured for private access, and S3 delivered a scalable and accessible object storage mechanism for the image files, configured with appropriate public read permissions via a bucket policy. These coordinated configurations and service integrations ultimately ensured that the deployed Photo Album system met the initial requirements. It was confirmed to be functional, accessible via its public endpoint, inherently scalable due to the nature of the chosen cloud services, and fundamentally secure through the implemented network and access control measures, thereby aligning with established cloud architecture best practices. The project thus serves as a practical demonstration of building and operating a well-architected application on the AWS platform.

12 REFERENCES:

[1] R. Buyya, J. Broberg, A. Goscinski, and J. Varia, *Best Practices in Architecting Cloud Applications in the AWS Cloud*. Hoboken, N.J. : Wiley, pp. 457–490. doi: 10.1002/9780470940105.ch18.

[2] D. Vohra, *Using the Amazon EC2*. [Berkeley, CA] : Apress, pp. 229–252. doi: 10.1007/978-1-4842-1830-3_15.

[3] S. Sahoo, A. Swain, S. K. Mohapatra, S. Pattanaik, and A. Senapati, "Utilizing AWS Cloud-Enhanced VPN and VPC Integration to Improve Data Security and Access Control in Library Management Systems," in 2024 Second International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI), IEEE, 2024, pp. 112–194. doi: 10.1109/ICoICI62503.2024.10696648.

[4] L. Narasimhan G, "Database Migration on Premises to AWS RDS," EAI endorsed transactions on cloud systems., vol. 3, no. 11, 2018, doi: 10.4108/eai.11-4-2018.154463.

[5] I. : (th : IEEE International Conference on Cloud Networking Pisa, "On the Network Performance of Amazon S3 Cloud-Storage Service," in CloudNet 2016 : proceedings : 2016 5th IEEE International Conference on Cloud Networking : 3-6 October 2016, Pisa, Italy., New York : IEEE, 2016, pp. 1–118. doi: 10.1109/CloudNet.2016.16.

[6] Greenspan, J., & Bulger, B. (2001). *MySQL/PHP Data Base Applications*. Prentice Hall.